1Q) i/p: This is vector institute (main string)

sub string → is

o/p: thsi si vector institute

2Q) i/p: This is vector institute

o/p: institute vector is this

1Q) declare a static int array
declare strrev function

in main:

1. i/p main string
   i/p Substring → call strrev fuc' here so that rev substring is available

while loop with strstr

2. Use strstr to find the Substring in main string

3. using memmove or strncpy Copy reverse Sub string into the actual address

memmove(p , rev , strlen(rev))

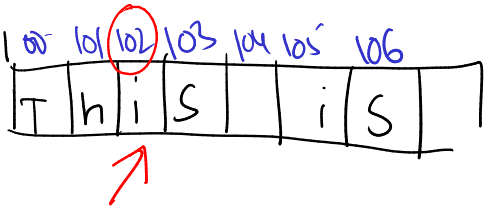
p → where substring is found

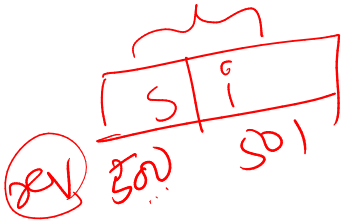
rev → static array

4. finally print the modified String.

pgm

```
#include <stdio.h>
char *rev[10];
void strrev(char *s);
main()
{
char m[50], s[10], *p;
gets(m);
gets(s);
strrev(s);  → p = m;
while (p = strstr(p, s))   → NULL
{
memcpy(p, rev, strlen(rev))
p = p + strlen(rev);
}
void strrev(char *s)
{
for(i=0, j= strlen(s)-1; s[i]; i++, j--)
{
rev[strlen(s)-1] = s[i];
}
}
```





20) Reversing of words

i/p this is vector - m
code

o/p vector is this m

1st step → rotcev si siht

reverse individual strings

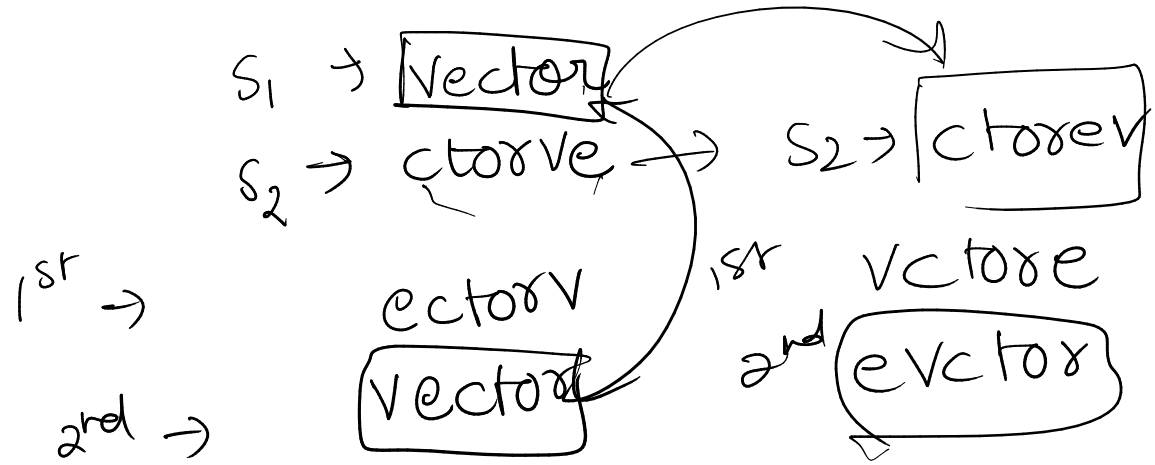
vector is this

Algorithm:

1. i/p string from user
2. Using loop reverse the whole string
3. Now run a loop again whenever found space reverse that particular word
4. This loop will continue till you reach NULL
5. finally main string with reverse words is present.

```
main()
{
    char s[50], *p, rev[50];
    gets(s);
    strrev(s);   // s is reversed
      p=s;
    p= strtok(s," ");
    {
      memmove(rev+strlen(rev), strrev(p), strlen(p));
        p = strtok(NULL," ");
    }
}
strcpy(s,rev);
```

59)



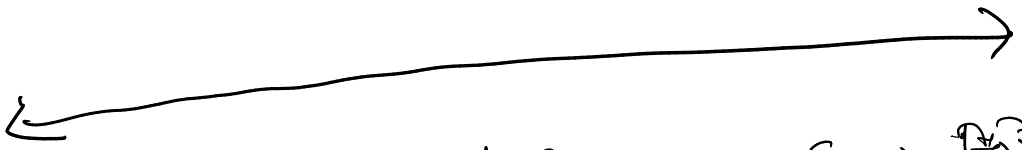
S_1

S_2 → rotate once circularly

compare S_1 & S_2 strcmp(S_1, S_2)

Algorithm:

1. i/p S1 & S2
2.



$S_1 \rightarrow$ vector $S_2 \rightarrow$ torvec

1st rotn of $S_2 \rightarrow$ ctorve $\rightarrow$ Strcmp(S_1, S_2) (non-zero)

2nd rotn of $S_2 \rightarrow$ ctorve $\rightarrow$ Strcmp(S_1, S_2) (non-zero)

3rd rotn of $S_2 \rightarrow$ vector $\rightarrow$ Strcmp(S_1, S_2) (0)

print (S_2 is circular permutation of S_1)

Algorithm:

1. i/p S1 & S2
2. run a loop (Strlength of S_1) times
3. In loop check for each rotation using Strcmp S1 & S2
4. If it returns 0 then print S2 is circular permutation of S1 & return 0 to main

5. If it comes out of loop print S_2 is not circular permutation of S1